CSE 472 Assignment 2

Decision Trees, Random Forests, and Extra Trees

Department of Computer Science and Engineering

Ananya Shahrin Promi
2005079

January 25, 2026

1 Introduction

Tree-based models are favored for their interpretability and efficiency with structured data. Ensemble techniques, specifically Random Forests and Extra Trees, enhance these models by aggregating multiple trees to improve robustness. This study involves implementing these three classifiers from scratch to compare their performance and algorithmic behavior against standard scikit-learn implementations, focusing on the impact of key hyperparameters.

2 Algorithmic Details

2.1 Custom Decision Tree

2.1.1 Algorithm

The Decision Tree (DT) is a binary CART-style classifier. Each internal node stores a feature index j and a threshold t. A sample $\mathbf{x}$ is routed left if $x_j \leq t$, and right otherwise. The best split is selected by minimizing a node impurity measure (Gini or Entropy).

2.1.2 Tree Construction and Split Strategy

- For each feature, all unique values in the node are candidate thresholds.
- Weighted impurity of the split is calculated:

$$I_{\text{split}} = \frac{|S_L|}{|S|} I(S_L) + \frac{|S_R|}{|S|} I(S_R)$$

  where $I(S)$ is either Gini(S) or Entropy.
- The split minimizing I_{split} is chosen.

2.1.3 Stopping Conditions

The node becomes a leaf if:

- Maximum depth (`max_depth`) is reached.

- Number of samples is smaller than `min_samples_split`.
- All samples belong to the same class.

Traverse the tree from root to leaf using the learned thresholds. Return the class label at the leaf node.

2.2 Custom Random Forest

2.2.1 Algorithm

Random Forest (RF) trains T decision trees on bootstrap samples of the training data. At each node, a random subset of features of size *max_features* is considered when selecting the best split. Predictions are obtained via majority voting:

$$\hat{y} = \text{mode}\{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T\}$$

2.2.2 Tree Construction and Split Strategy

- For each tree, draw a bootstrap sample of size N from the training set.
- Train a Decision Tree on the sample. Only randomly selected features are used at each split.
- Select the split minimizing node impurity (Gini or Entropy).

Each tree predicts independently. The forest prediction is the majority vote over all trees.

2.3 Custom Extra Trees

2.3.1 Algorithm

Extra Trees (ET) is an ensemble method similar to Random Forest but introduces extra randomness by selecting random thresholds for candidate features instead of searching all possible splits. Trees are typically trained on the full dataset without bootstrapping. Predictions are aggregated via majority voting.

2.3.2 Tree Construction and Split Strategy

- At each node, select a random subset of features of size *max_features*.
- For each feature, sample a random threshold uniformly in the feature's range:

$$t_j \sim \mathcal{U}(\min(x_j), \max(x_j))$$

- Evaluate impurity (Gini or Entropy) for the split and choose the best one.
- Recursively grow the tree until stopping conditions are met.

2.3.3 Stopping Conditions

- Maximum depth reached.
- Node contains fewer than `min_samples_split` samples.
- Node is pure (all samples belong to one class).

Predictions from all trees are aggregated by majority vote to produce the final output.

3 Experimental Setup

3.1 Datasets

We evaluated our models on two benchmark datasets:

- **Iris Dataset:** Consists of 150 samples with 4 features each, divided into 3 classes.
- **Wine Dataset:** Contains 178 samples with 13 features and 3 classes.

3.2 Train-Test Split

- Each dataset was split into training and testing sets using an 80:20 ratio.
- Stratified splitting was used to preserve class distributions in both sets.

3.3 Model Parameters

Decision Tree:

- Maximum depth, `max_depth` = None
- Minimum samples per split, `min_samples_split` = 2

Random Forest:

- `max_depth` = None
- `min_samples_split` = 2
- Number of estimators, `n_estimators` = 500
- `max_features` = $\sqrt{F}$, where F is the number of features

Extra Trees:

- `max_depth` = None
- `min_samples_split` = 2
- `n_estimators` = 500
- `max_features` = $\sqrt{F}$

3.4 Evaluation Metrics

- **Accuracy:** Fraction of correctly predicted samples.
- **Macro F1-Score:** F1-Score averaged across all classes without weighting by class size.
- **AUROC (Area Under Receiver Operating Characteristic Curve):** Computed in a one-vs-rest fashion for multi-class datasets.

4 Results and Analysis

We evaluate all models on the Iris and Wine datasets using three metrics. Results are reported for both our custom implementations and scikit-learn implementations.

4.1 Performance Tables

Model	Accuracy	Macro F1	AUROC
Custom Decision Tree	0.9333	0.9333	0.9500
sklearn Decision Tree	0.9333	0.9333	0.9500
Custom Random Forest	0.9667	0.9666	0.9750
sklearn Random Forest	0.9333	0.9333	0.9500
Custom Extra Trees	0.9667	0.9666	0.9750
sklearn Extra Trees	0.9667	0.9666	0.9750

Table 1: Model Performance on Iris Dataset

Model	Accuracy	Macro F1	AUROC
Custom Decision Tree	0.9167	0.9230	0.9408
sklearn Decision Tree	0.9444	0.9457	0.9543
Custom Random Forest	1.0000	1.0000	1.0000
sklearn Random Forest	1.0000	1.0000	1.0000
Custom Extra Trees	1.0000	1.0000	1.0000
sklearn Extra Trees	1.0000	1.0000	1.0000

Table 2: Model Performance on Wine Dataset

4.2 Visualization of Results

- **Grouped Bar Charts:**

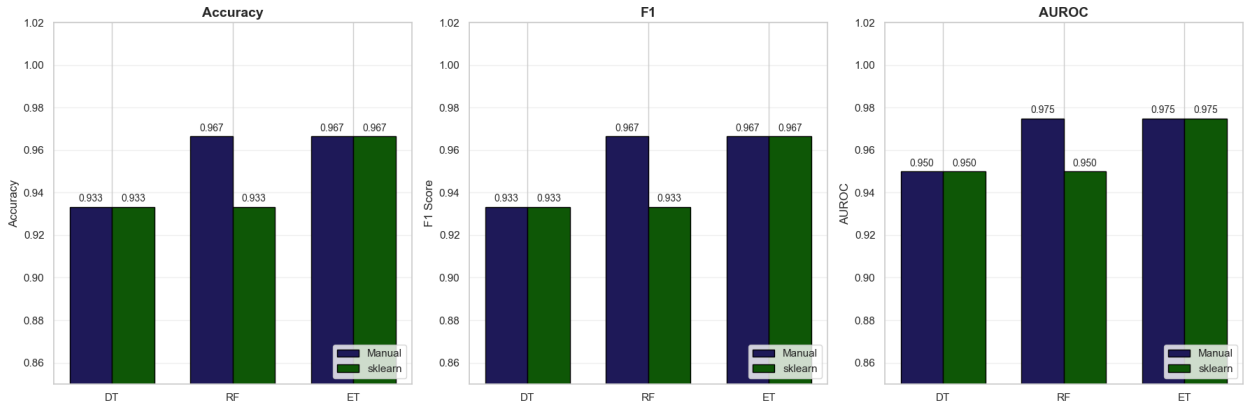

Figure 1: Grouped bar charts comparing Manual vs scikit-learn implementations on Iris dataset

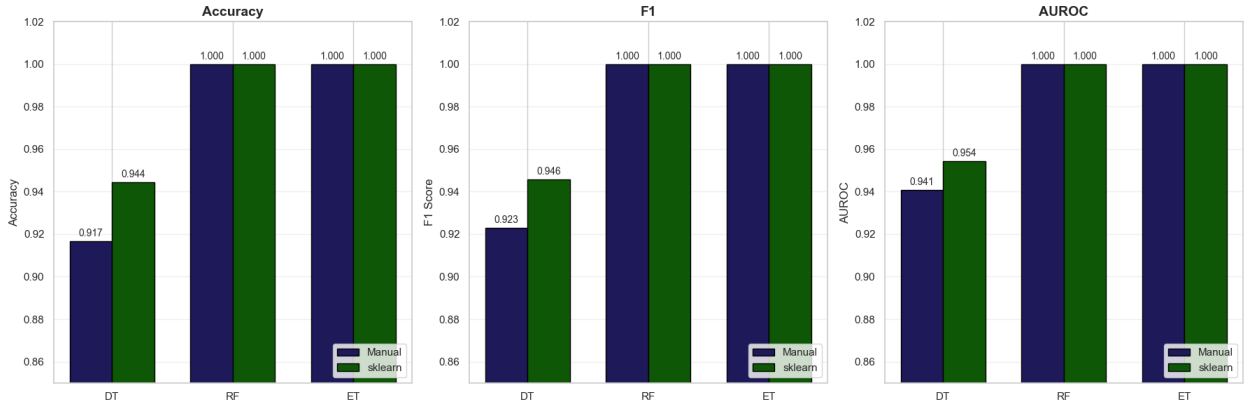

Figure 2: Grouped bar charts comparing Manual vs scikit-learn implementations on Wine dataset

- **Heatmaps:**

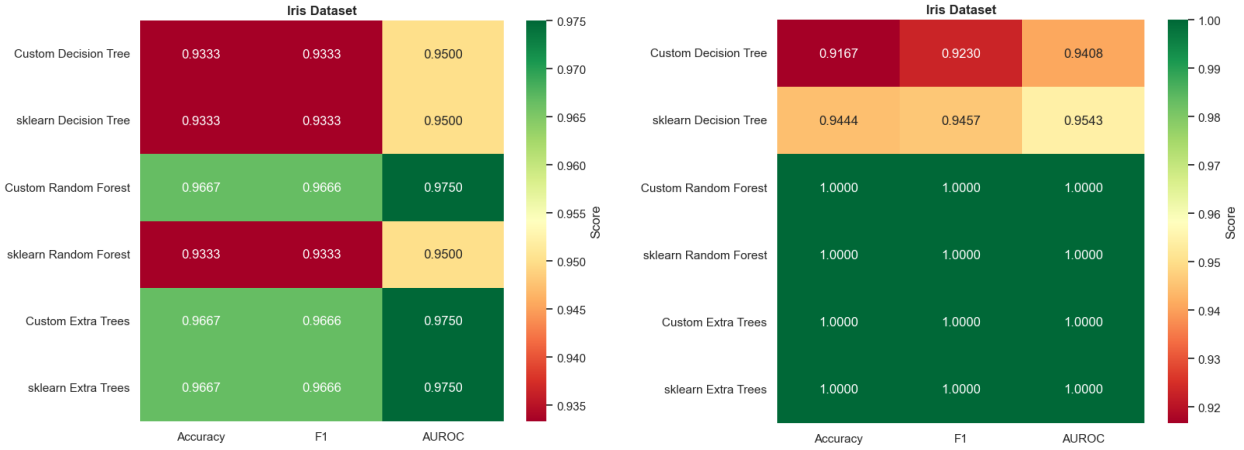

Figure 3: Heatmap comparison

- **Radar Charts:**

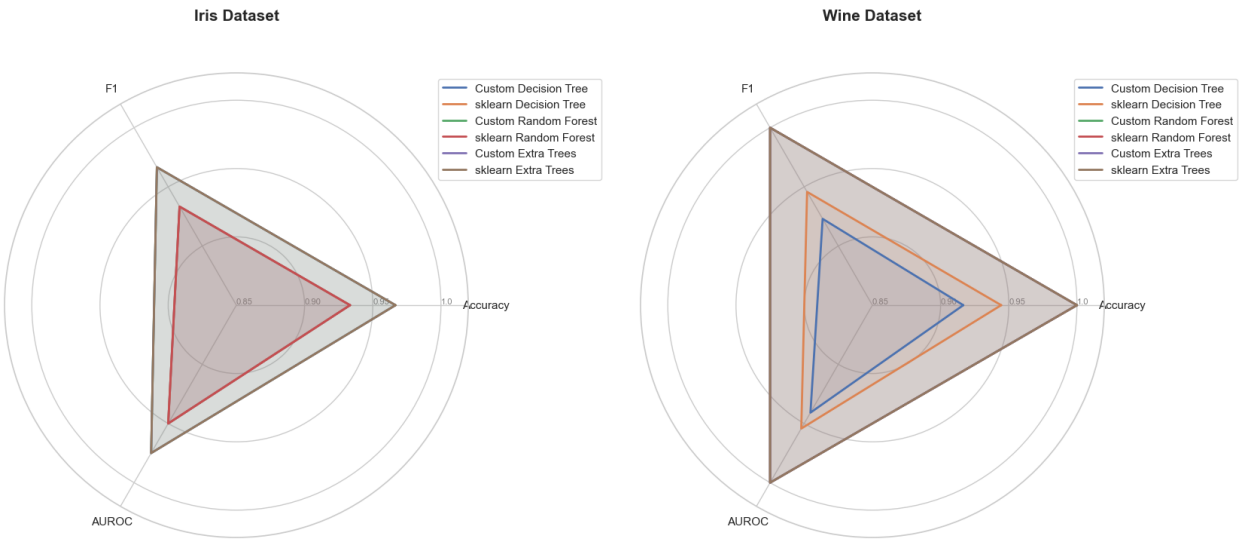

Figure 4: Radar Chart comparison

4.3 Analysis

- **Decision Trees:** The manual implementation closely matches the scikit-learn results on the Iris dataset, while showing slightly lower performance on Wine. This indicates that the core splitting logic is correct, with minor differences likely arising from internal tie-breaking and implementation details.

- **Random Forests:** Ensemble methods consistently outperformed single decision trees. The custom Random Forest achieved strong performance on Iris, comparable to or exceeding the scikit-learn baseline, while both implementations reached near-perfect performance on the Wine dataset.

- **Extra Trees:** Extra Trees demonstrated high robustness, achieving performance comparable to Random Forests across both datasets. The use of random threshold selection effectively reduced variance while preserving strong predictive capability.

- **Hyperparameters:** Performance reached a saturation point as n_trees increased, while optimal max_depth was critical to balancing the underfitting/overfitting trade-off.

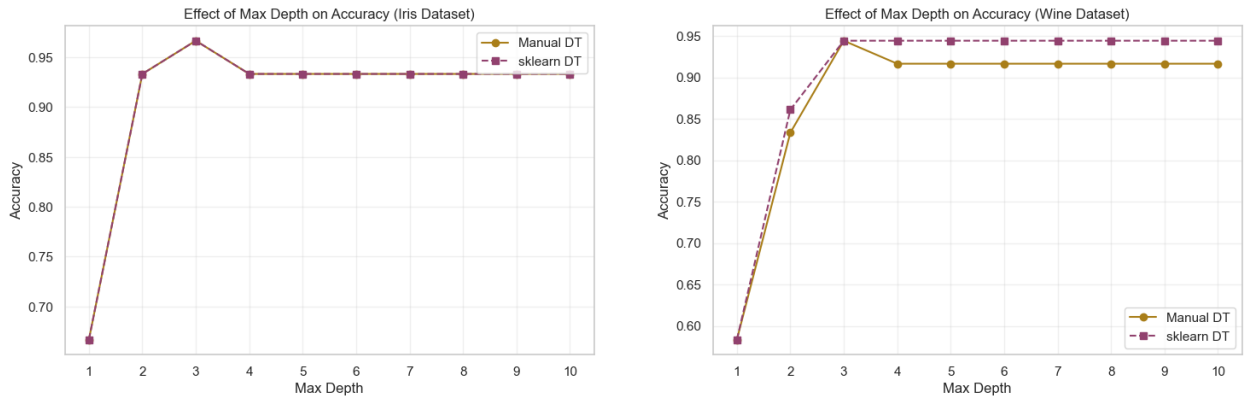

Figure 5: Effect of max depth on Decision Tree accuracy comparison

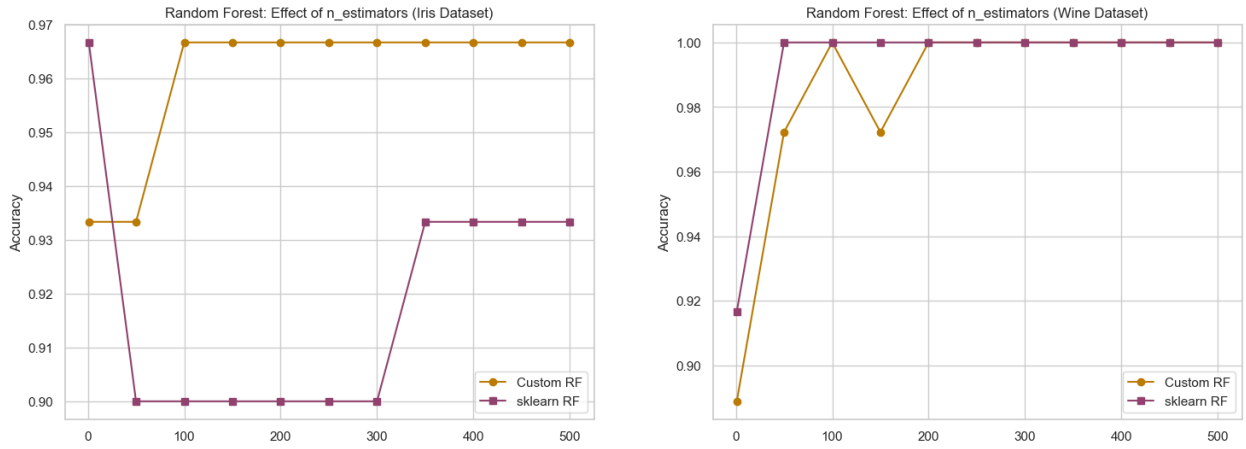

Figure 6: Effect of n_trees on Random Forest accuracy comparison

5 Conclusion

The strong alignment with scikit-learn benchmarks across all evaluation metrics validates the correctness of the implemented bagging, feature subsampling, and randomization strategies.